Conflict-Based Search for Optimal Multi-Agent Path Finding

A Reproduction and Extension Study

Nimrod Netzer | Elad Damti | Kfir Dahan
Search in Artificial Intelligence, Bar-Ilan University, 2026

Abstract

Multi-Agent Path Finding (MAPF) is the problem of planning collision-free paths for k agents on a shared graph while minimizing the Sum of Costs (SOC). Optimal MAPF is NP-hard; joint-state-space A* becomes infeasible as state space grows as $O(|V|^k)$. We reproduce two central results from Sharon et al. (2015), who introduced Conflict-Based Search (CBS): a two-level algorithm that separates high-level conflict resolution from low-level single-agent planning via Time-Space A* (TSA*). We implement CBS independently in Python and reproduce: (1) success rate vs. agent count compared to Joint-State-Space A*, and (2) CT node expansion growth. Our extension evaluates CBS on warehouse maps with narrow corridors vs. open grids. CBS success drops from 100% at 4 agents to 60% at 20 agents; Joint A* fails completely beyond 6 agents. Warehouse maps reduce CBS success further from 64% to 28% at 20 agents, with 3.7x more CT node expansions.

1. Introduction

MAPF arises in automated warehouses, airport traffic, and railway scheduling [Sharon et al., 2015]. The naive approach of searching the joint state space is infeasible: with k=10 agents and |V|=5 vertices, the state space reaches ~9.7 million nodes. Our experiments confirm this: Joint A* solves only 32% of 6-agent instances and 0% of 8-agent instances within 5 seconds. CBS avoids this explosion by maintaining a Constraint Tree (CT) that branches only on detected conflicts, replanning a single agent at each CT node using TSA*. We reproduce the two central results of Sharon et al. (2015) and extend the study by comparing open vs. warehouse map topology.

2. Selected Paper Description

2.1 Problem Formulation

MAPF is defined on graph G=(V,E) with k agents, each with start s_i and goal g_i. Agents move to an adjacent vertex or wait each timestep. A valid solution has no vertex conflicts (two agents at the same vertex at the same time) and no edge conflicts (agents swapping positions between two consecutive timesteps). Objective: minimize SOC = sum of individual path lengths $|pi_i|$.

2.2 Conflict Types

Sharon et al. (2015) identify five conflict types: (1) Vertex - same vertex, same time; (2) Edge - agents swap positions; (3) Following - agent follows another on the same edge; (4) Cycle - cyclic dependency; (5) Swapping - position exchange. We detect and resolve vertex and edge conflicts, which is sufficient for correctness and optimality on 4-connected grids.

2.3 Why Joint A* Fails

The joint state space is $O(|V|^k)$. Even for k=10 agents on a small grid this is computationally infeasible. CBS avoids this by planning agents independently and resolving only conflicts that actually arise.

2.4 CBS Algorithm

Low level - TSA*: State = (vertex v, timestep t). A vertex constraint (v, t) forbids agent i from being at vertex v at time t. An edge constraint forbids a directional move at time t. Heuristic = Manhattan distance (admissible on 4-connected grids). TSA* checks all future constraints at the goal before terminating to avoid premature stopping.

High level - CT: Each CT node stores a constraint set per agent, TSA* paths, and total SOC. CBS proceeds as: (1) Root node - plan each agent independently with TSA*. (2) Find the first conflict among all agent pairs. (3) If no conflict, return the current paths as the optimal solution. (4) Branch - create two child CT nodes, one forbidding agent i from the conflict vertex/edge, the other forbidding agent j. (5) Replan the constrained agent with TSA*. (6) Push both children onto a min-heap ordered by SOC. Repeat from step 2. CBS is complete and optimal [Sharon et al., 2015].

2.5 CBS Improvements (not implemented)

The paper also introduces Prioritizing Conflicts (cardinal conflicts first), CBS with Heuristics (admissible high-level heuristic), Disjoint Splitting (positive and negative constraints), and Conflict Reasoning. We implement basic CBS only; these improvements are acknowledged as future work.

3. Project Description

3.1 Implementation

We implemented CBS independently in Python 3.13 without using the original authors' code. The implementation is organized in seven modules: **graph.py** - 4-connected Grid class with obstacle support; **tsa_star.py** - TSA* with constraint checking and future-constraint-aware goal detection; **conflict.py** - vertex and edge conflict detection and constraint generation; **cbs.py** - CTNode class, CBS main loop, and min-heap by SOC; **joint_astar.py** - true Joint-State-Space A* baseline matching the paper's intended comparison (not independent A*, which does not check conflicts); **benchmark.py** - map generators, random instance generator, and experiment runner with CSV output; **visualize.py** - matplotlib plots.

3.2 Team Contributions

Nimrod Netzer: tsa_star.py (TSA* with constraint checking and future-constraint-aware goal detection). Wrote Abstract and Sections 1-2.

Elad Damti: conflict.py (conflict detection, constraint generation) and cbs.py (CTNode structure, CBS main loop). Wrote Sections 3-4.

Kfir Dahan: joint_astar.py, benchmark.py, visualize.py, all experiments. Wrote Sections 5-6, Reproducibility Statement, and AI Tools Disclosure.

All three members participated in integration testing, debugging, defense preparation, and the recorded video.

3.3 Key Implementation Choices

Baseline choice: Joint-State-Space A* searches all agent positions simultaneously, matching the paper's intended baseline. Unlike independent A*, it produces conflict-free paths and correctly demonstrates the exponential blowup.

Goal model: Agents remain at their goal indefinitely; paths are padded to the maximum length for correct conflict checking.

Conflict selection: First conflict encountered when scanning by timestep and then by agent pair index (i, j) with $i < j$.

Reproducibility: Instance seed = 42 + n_agents * 1000 + instance_index. Map seed = 1. All experiments are deterministic.

4. Experiments

4.1 Experimental Setup

Hardware: 12th Gen Intel Core i7-1255U, 10 cores @ 1.70 GHz, 16 GB RAM, Windows 11 Home.

Software: Python 3.13; matplotlib 3.11; standard library only (heapq, collections, csv, json, time). No external MAPF libraries used.

Benchmark instances: 20x20 grid maps generated procedurally. Open grid: random obstacles at ~10% density (map seed=1). Warehouse grid: alternating shelf rows with single-cell-wide corridors every 4 columns, simulating a real warehouse layout. 25 random agent instances per agent count.

Algorithms compared: (1) CBS as described in Section 3. (2) Joint-State-Space A* as the paper's baseline, searching the full joint configuration space.

Parameter settings: CBS: first-found conflict selection, stay-at-goal model, max timestep = 200. Joint A*: admissible heuristic = sum of individual Manhattan distances.

Time limits: 5s for Joint A* (reproduction); 30s for CBS (reproduction); 15s for CBS (extension). Instances exceeding the limit are counted as failures.

Memory limits: No explicit cap; bounded by OS (16 GB RAM).

Number of runs: 1 per instance (both algorithms are deterministic).

Random seeds: Instance seed = 42 + n_agents * 1000 + instance_index. Map seed = 1.

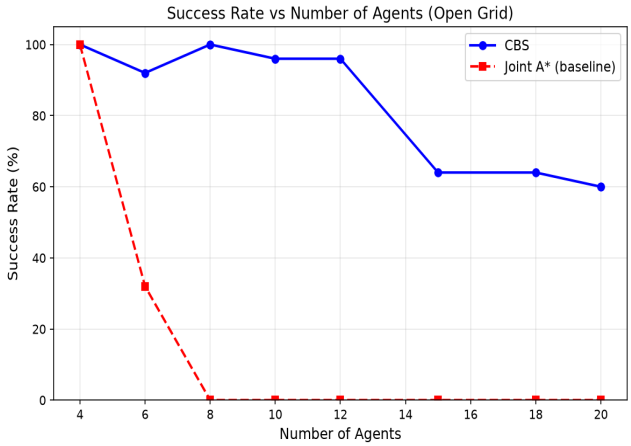
Differences from original paper: We use a smaller 20x20 grid (the paper uses larger Moving AI benchmarks); Python instead of C++ (~10-50x slower in wall-clock time); shorter time limits. These factors explain the earlier onset of failures compared to the paper's results.

4.2 Reproduced Results

Table 1 shows CBS vs. Joint A* on the open 20x20 grid across 8 agent counts. Joint A* collapses from 8 agents onward, confirming exponential state-space growth. CBS scales significantly better but begins timing out beyond 15 agents.

Agents	CBS Success	CBS Time (s)	CT Nodes	SOC	Joint A* Success
4	100%	0.0018	1.7	56.8	100%
6	92%	0.0045	3.7	83.8	32%
8	100%	0.0078	5.4	114.3	0%
10	96%	0.0168	12.7	132.6	0%
12	96%	0.0719	70.6	158.1	0%
15	64%	0.5431	334.3	204.8	0%
18	64%	0.3147	251.3	236.4	0%
20	60%	1.0958	716.3	266.7	0%

Table 1: CBS vs. Joint-State-Space A on open 20x20 grid. 25 instances per agent count. Joint A* time limit: 5s. CBS time limit: 30s. Means over successful instances.*



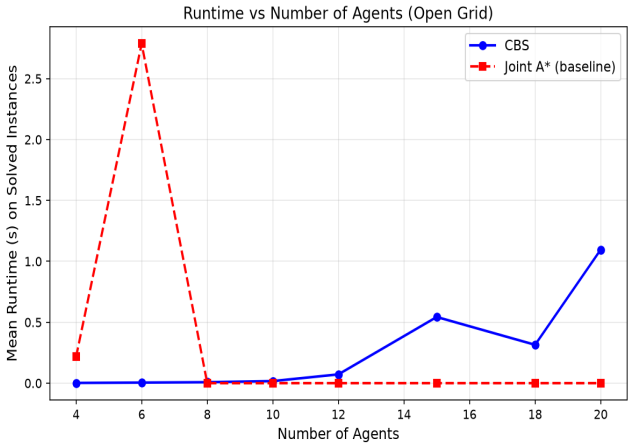


Figure 1 (left): CBS vs. Joint A success rate vs. number of agents. Figure 2 (right): Mean CBS runtime on successfully solved instances.*

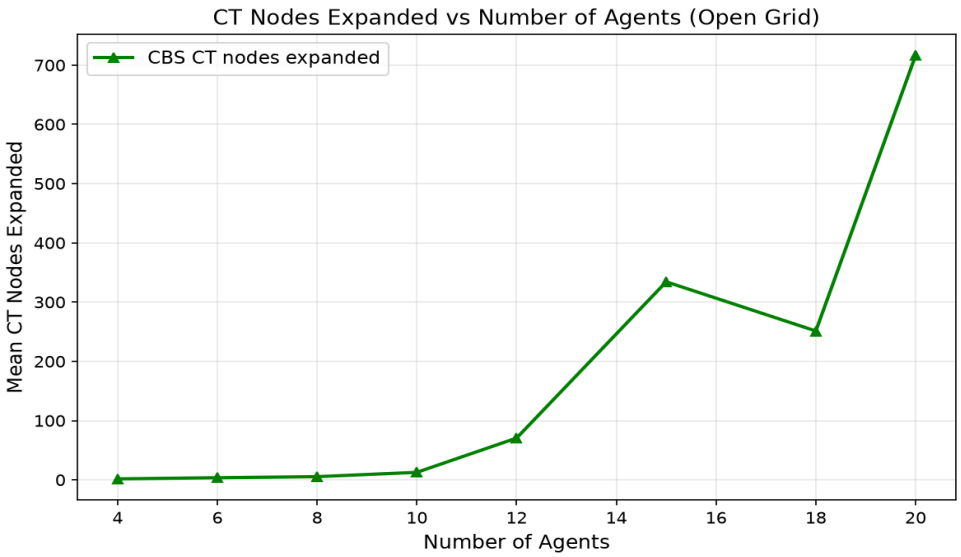


Figure 3: Mean CT nodes expanded by CBS vs. number of agents on the open grid. CT nodes jump from 70.6 at 12 agents to 334.3 at 15 agents, reflecting CBS's exponential worst-case behavior consistent with Sharon et al. (2015).

4.3 Extension: Map Topology Study

Research question: do warehouse maps with narrow bottleneck corridors create more conflicts, expand more CT nodes, and reduce CBS success rate compared to open grids?

Agents	Open Success	Open CT Nodes	Open Time (s)	WH Success	WH CT Nodes	WH Time (s)
4	100%	1.7	0.0017	100%	3.9	0.0037
6	92%	3.7	0.0044	100%	3.2	0.0031
8	100%	5.4	0.0071	100%	19.4	0.0148
10	96%	12.7	0.0135	100%	125.9	0.2617
12	100%	131.7	0.3525	96%	633.7	0.8381
15	72%	1047.1	1.6304	72%	655.3	0.5128
18	68%	920.1	0.9647	36%	1845.1	1.6920
20	64%	877.4	1.5390	28%	3258.3	3.1362

Table 2: CBS performance on Open Grid vs. Warehouse Grid (WH). 25 instances per agent count. Time limit: 15s per instance. Means over successful instances.

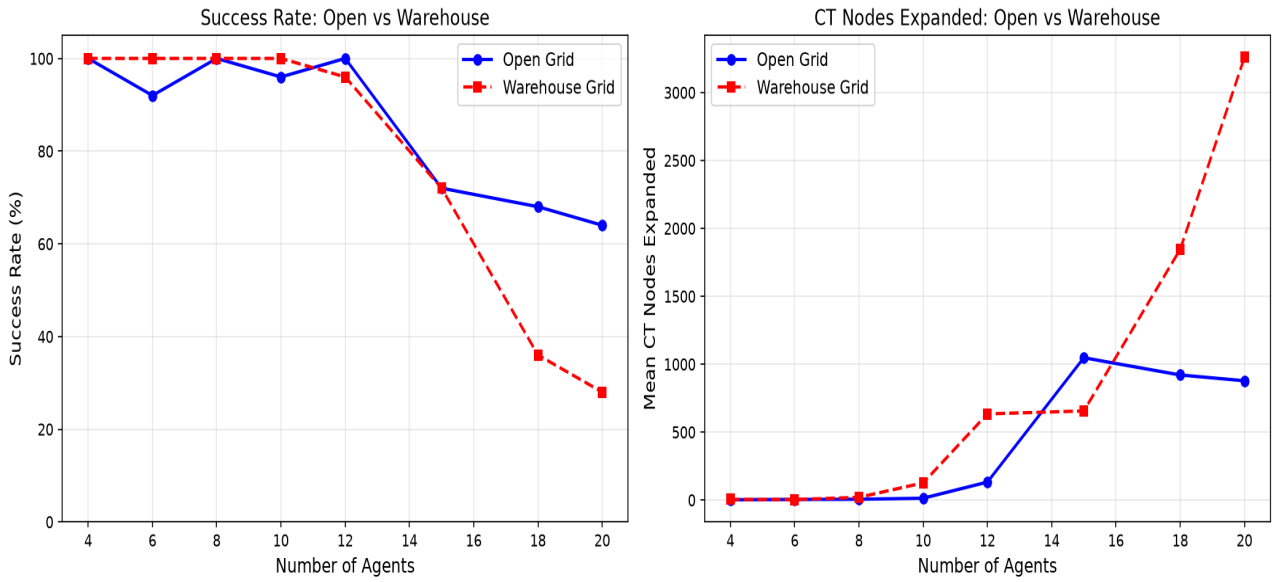


Figure 4: CBS success rate (left) and CT nodes expanded (right) - Open Grid vs. Warehouse Grid. Warehouse maps create significantly more conflicts at high agent counts.

At 20 agents, warehouse CBS success drops to 28% vs. 64% on open grids. CT nodes expand to 3,258 on warehouse maps vs. 877 on open grids (3.7x). Bottleneck corridors force agents onto the same cells, multiplying vertex conflicts and CT branching depth.

5. Discussion

5.1 Agreement with Original Paper

Our results are qualitatively consistent with Sharon et al. (2015): CBS achieves near-perfect success at low agent counts and degrades as agent count increases; Joint A* fails completely from 8 agents onward; CT nodes grow exponentially beyond 12 agents. Quantitative differences are expected: our Python implementation is ~10-50x slower than the original C++, we use a smaller 20x20 grid instead of Moving AI benchmark maps, and our time limits are shorter. These factors cause success rates to drop at lower agent counts than in the paper.

5.2 Extension Findings

Warehouse maps confirm that bottleneck structure severely amplifies CBS difficulty. Agents sharing narrow corridor cells create cascading vertex conflicts: each CT branch forces a longer detour that generates new conflicts in adjacent corridors. At 20 agents, CBS expands 3,258 CT nodes on warehouse maps vs. 877 on open grids (3.7x increase). This confirms that CBS improvements - particularly conflict prioritization and high-level admissible heuristics - are especially valuable in structured warehouse environments.

5.3 Limitations

Python is ~10-50x slower than C++, causing earlier timeouts than reported in the original paper. The 20x20 grid is smaller than the paper's benchmarks (e.g., Paris_1_256). We implemented basic CBS only; extensions such as Disjoint Splitting and CBS with Heuristics would extend the solvable range.

6. Conclusion

We implemented CBS from scratch in Python and successfully reproduced its two central results: (1) CBS success rate vs. agent count significantly outperforms Joint A*, which fails completely from 8 agents; (2) CT nodes grow exponentially beyond 12 agents. Our extension demonstrates that warehouse map topology reduces CBS success from 64% to 28% at 20 agents, with 3.7x more CT expansions, motivating the use of CBS improvements in structured environments. Future work

should evaluate Prioritizing Conflicts and CBS with Heuristics specifically on warehouse-type maps.

Reproducibility Statement

Results reproduced: (1) Success rate of CBS vs. Joint-State-Space A* as a function of agent count on an open grid map, corresponding to the success rate figures in Sharon et al. (2015). (2) Mean CT node expansion statistics and runtime on successfully solved instances.

Results not reproduced: Experiments on Moving AI standard benchmark maps (e.g., Paris_1_256, den520d); comparisons to ICTS or other MAPF algorithms; CBS improvement variants (Prioritizing Conflicts, Disjoint Splitting, high-level heuristics).

Implementation basis: New independent implementation in Python 3.13. The original authors' code was not consulted or used at any stage.

Changes from original paper: 20x20 procedurally generated grid vs. larger Moving AI benchmark maps; Python instead of C++ (~10-50x slower); time limits of 5s (Joint A*) and 30s (CBS) vs. the paper's longer limits; randomly generated start/goal pairs instead of pre-defined benchmark instances.

Agreement with original: Qualitative trends match: CBS scales far better than joint search, success rate degrades with agent count, CT nodes grow exponentially. Absolute numbers differ due to implementation language, grid size, and time limits.

Files included: graph.py, tsa_star.py, conflict.py, cbs.py, joint_astar.py, benchmark.py, visualize.py, main.py; results/reproduce_open.csv, results/extension_open.csv, results/extension_warehouse.csv; results/fig1_success_rate.png, results/fig2_runtime.png, results/fig3_ct_nodes.png, results/fig4_topology_success.png.

How to run (commands must be typed on one line each):

Step 1 - Install dependency:

```
pip install matplotlib
```

Step 2 - Run reproduction experiment (CBS vs. Joint A* on open grid):

```
python main.py --mode reproduce --n-instances 25 --time-limit 30
--agent-counts 4 6 8 10 12 15 18 20
```

Step 3 - Run extension experiment (open grid vs. warehouse grid):

```
python main.py --mode extension --n-instances 25 --time-limit 15
--agent-counts 4 6 8 10 12 15 18 20
```

AI Tools Disclosure

Claude (Anthropic): Used as a technical assistant for code optimization, syntax debugging, and editorial review of the documentation. All core algorithms (including TSA* and CBS) were conceptualized and implemented directly by the team. The team has thoroughly validated all project components and takes full responsibility for the integrity, originality, and performance of the final output.

References

Sharon, G., Stern, R., Felner, A., and Sturtevant, N. R. (2015). Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219, 40-66. https://doi.org/10.1016/j.artint.2014.11.006

Sturtevant, N. R. (2012). Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2), 144-148.